

Ranking in RMAB

Team: BetaStay

Topics in Reinforcement Learning

Proposed Approach: Ranking-based Actor–Critic Policy

Key Idea:

- Model RMAB scheduling as a **learned ranking problem**
- Instead of selecting actions directly, learn a **score per arm**
- Convert scores into a **top- M stochastic ranking policy**

Core Architecture:

- **Actor Network** $f_{\theta}(x_i^t)$:
- **Critic Network** $V_{\psi}(s_t)$:

Policy Construction:

- Scores $\rightarrow$ Plackett–Luce ranking distribution
- Sample top- M arms using Gumbel-top- M
- Produces stochastic but structured scheduling policy

Core Idea: Learn a Score per Arm

Step 1: Per-arm encoding

For each arm i with state x_i^t :

$$s_i^t = f_\theta(x_i^t)$$

Interpretation:

- s_i^t = “priority / urgency score”
- Computed using a shared neural network
- Same function applied to all arms

Output of actor:

$$\{s_1^t, s_2^t, \dots, s_N^t\}$$

Action Selection Problem

Goal:

- Select M arms out of N
- But selection must be:
 - stochastic (exploration)
 - differentiable (for learning)

Naive idea (fails):

- Take top- M scores:

$$\text{TopM}(s_1^t, \dots, s_N^t)$$

- Not differentiable $\Rightarrow$ no gradient flow

Solution idea:

- Convert scores into a probability distribution
- Sample M arms without replacement
- Use **Plackett-Luce ranking model**

Plackett-Luce Ranking Policy (Key Idea)

Step 2: Sampling procedure:

- Add Gumbel noise to scores

$$\tilde{s}_i = s_i + g_i, \quad g_i \sim \text{Gumbel}(0, 1)$$

- Sort perturbed scores
- Take top- M (our ranking now)

Convert scores into a ranking distribution

$$P(a_1, a_2, \dots, a_M \mid s) = \prod_{k=1}^M \frac{\exp(s_{a_k})}{\sum_{j \notin \{a_1, \dots, a_{k-1}\}} \exp(s_j)}$$

Gumbel-Max Trick and Plackett–Luce Sampling

Why does Gumbel work?

Gumbel noise is generated as

$$g_i = -\log(-\log u_i), \quad u_i \sim \text{Uniform}(0, 1)$$

Sampling rule

$$a = \arg \max_i (\tilde{s}_i) = \arg \max_i (s_i + g_i)$$

Probability of selecting arm i

$$P(a = i) = P(s_i + g_i \geq s_j + g_j, \forall j)$$

Rearranging,

$$P(a = i) = P(g_j \leq g_i + s_i - s_j, \forall j)$$

Using the Gumbel CDF,

$$F(g) = \exp(-e^{-g})$$

and independence of noises,

$$\begin{aligned} P(a = i \mid g_i) &= \prod_{j \neq i} \exp\left(-e^{-(g_i + s_i - s_j)}\right) \\ &= \exp\left(-e^{-g_i} \sum_{j \neq i} e^{s_j - s_i}\right) \end{aligned}$$

Integrating over g_i yields

$$P(a = i) = \frac{\exp(s_i)}{\sum_j \exp(s_j)}$$

Conclusion

Adding i.i.d. Gumbel noise and taking the argmax is exactly equivalent to sampling from a softmax distribution.

Sequential sampling without replacement gives the Plackett-Luce ranking

Why is

$$P(a_1, a_2, \dots, a_M \mid s)$$

a valid probability?

For each selection step k ,

$$P(a_k \mid a_1, \dots, a_{k-1}) = \frac{\exp(s_{a_k})}{\sum_{j \notin \{a_1, \dots, a_{k-1}\}} \exp(s_j)}$$

Since $\exp(s_j) > 0$ for all j ,

$$\sum_{j \notin \{a_1, \dots, a_{k-1}\}} \exp(s_j) \geq \exp(s_{a_k})$$

since the summation on LHS also contains $\exp(s_{a_k})$. Thus, denominator always greater than equal to numerator. hence it is a valid prob.

distribution. Hence every conditional selection probability is valid, and the overall ranking probability

$$P(a_1, \dots, a_M \mid s) = \prod_{k=1}^M P(a_k \mid a_1, \dots, a_{k-1})$$

Derivation (Plackett–Luce):

Using Luce's choice axiom, the probability of selecting arm i from a candidate set A is

$$P(i \mid A) = \frac{\exp(s_i)}{\sum_{j \in A} \exp(s_j)}$$

- Select a_1 from all arms

$$P(a_1) = \frac{\exp(s_{a_1})}{\sum_j \exp(s_j)}$$

- Remove a_1 , then select a_2

$$P(a_2 \mid a_1) = \frac{\exp(s_{a_2})}{\sum_{j \neq a_1} \exp(s_j)}$$

- Continue similarly up to a_M

Applying the chain rule:

$$P(a_1, \dots, a_M) = \prod_{k=1}^M P(a_k \mid a_1, \dots, a_{k-1})$$

Critic + Advantage for Ranking Policy

Critic (Value Function): Evaluates whether a sampled ranking is good or bad.

$$s_t := (x_1^t, x_2^t, \dots, x_N^t)$$

$$V(s_t) = \mathbb{E}[\text{return from } s_t]$$

One-step rollout target:

- At step t , actor produces a ranking and selects action
- Transition to next state s_{t+1}

$$y_t = r_t + \gamma V(s_{t+1})$$

Critic update (TD-style):

$$\mathcal{L}_{critic} = (V(s_t) - (r_t + \gamma V(s_{t+1})))^2$$

Advantage Signal for Ranking Policy

Advantage of sampled ranking:

$$A_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Interpretation:

- $A_t > 0 \Rightarrow$ ranking better than expected
- $A_t < 0 \Rightarrow$ ranking worse than expected

Policy update (actor):

$$\nabla_{\theta} J(\theta) = \mathbb{E} [A_t \nabla_{\theta} \log P_{\theta}(a_{1:M} \mid s_t)]$$

Improve the policy parameters θ in the direction that increases the log-probability of good actions (rankings), scaled by how good they were (advantage).

Policy update (actor):

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s_t \sim d^{\pi_{\theta}}, a_{1:M} \sim P_{\theta}(\cdot | s_t)} [A_t \nabla_{\theta} \log P_{\theta}(a_{1:M} | s_t)]$$

where:

- $d^{\pi_{\theta}}(s_t)$: state distribution induced by the current policy
- $P_{\theta}(a_{1:M} | s_t)$: probability of generating a ranked list (Plackett–Luce policy)
- $A_t = R_t - V_{\psi}(s_t)$: advantage function (baseline-corrected return)

Interpretation:

- The expectation is over trajectories induced by the policy: states s_t , sampled rankings $a_{1:M}$, and environment stochasticity.
- If $A_t > 0$, increase probability of the sampled ranking: $P_{\theta}(a_{1:M} | s_t)$.
- If $A_t < 0$, decrease its probability.
- This performs gradient ascent on expected return under the current policy.

Policy Gradient Foundation

$$\nabla_{\theta} J(\theta) = \mathbb{E} [A_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

Problem:

- No restriction on how much π_{θ} changes per update
- Can lead to large policy shifts and collapse

PPO Objective (How it fixes the problem)

Step 1: Probability ratio

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

- $r_t = 1$: no change
- $r_t > 1$: action becomes more likely
- $r_t < 1$: action becomes less likely

Step 2: PPO objective

$$L^{\text{PPO}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right]$$

What clipping does:

- Prevents probability ratio from changing too much
- Stops over-updating even if advantage is large

Full Training Objective

Overall loss function:

$$L = L_{\text{actor}} + \lambda_c L_{\text{critic}} + \lambda_m L_{\text{margin}} - \lambda_e H$$

This objective jointly optimizes:

- policy improvement (actor)
- value estimation (critic)
- ranking structure (margin)
- exploration (entropy)

Full Objective + Optimization

Total loss:

$$L = L_{\text{actor}} + \lambda_c L_{\text{critic}} + \lambda_m L_{\text{margin}} - \lambda_e H$$

1. Actor loss

$$L_{\text{actor}} = -\mathbb{E} \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right]$$

2. Critic loss

$$L_{\text{critic}} = \mathbb{E} \left[\left(V_{\psi}(s_t) - (r_t + \gamma V_{\psi}(s_{t+1})) \right)^2 \right]$$

3. Margin loss

$$L_{\text{margin}} = \mathbb{E} [\max(0, \Delta - (s_i - s_j))]$$

4. Entropy bonus

$$H = -\mathbb{E} \left[\sum_a \pi_{\theta}(a | s_t) \log \pi_{\theta}(a | s_t) \right]$$

Deadline Scheduling (RMAB Setup)

Problem motivation: EV charging stations

- A charging station has N charging slots (arms)
- At each round, only M vehicles can be charged (active arms)

State of each arm (vehicle):

$$s = (D, B)$$

- D : remaining time until deadline (max 12)
- B : remaining uncharged energy demand (max 9)

System Dynamics (EV Charging RMAB)

If an arm is **ACTIVATED** (charged):

- It receives 1 unit of service (or fractionally reduced demand)
- Demand reduces:

$$B \leftarrow \max(B - 1, 0)$$

- Immediate reward:

$$(1 - c)$$

per unit of charge delivered

If an arm is **NOT** activated (idle):

- No service is given $\rightarrow$ no immediate reward
- Deadline decreases:

$$D \leftarrow D - 1$$

When deadline expires ($D = 0$):

- Vehicle leaves the system
- Remaining demand B is unmet
- Station pays penalty: $F(B)$

Why this is an RMAB

Restless Multi-Armed Bandit structure:

- Each vehicle = one arm with Markov state (D, B)
- At each round, only M arms can be activated
- All arms evolve over time, whether activated or not (“restless”)

Key properties:

- Activated arms: gain reward + reduce demand
- Passive arms: age (deadline decreases)
- Final penalty couples time and remaining demand

Objective:

$$\max \mathbb{E} \left[\sum (1 - c) \cdot \text{service} - F(B_{\text{exit}}) \right]$$

Results

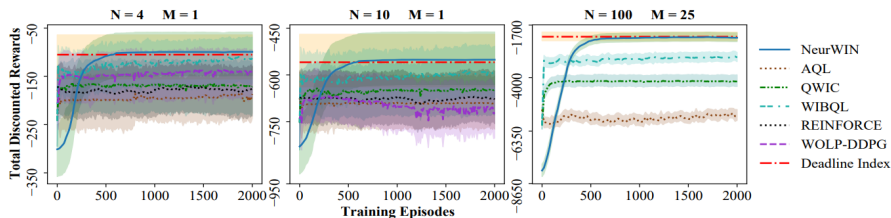


Figure: NeurWIN results

Results

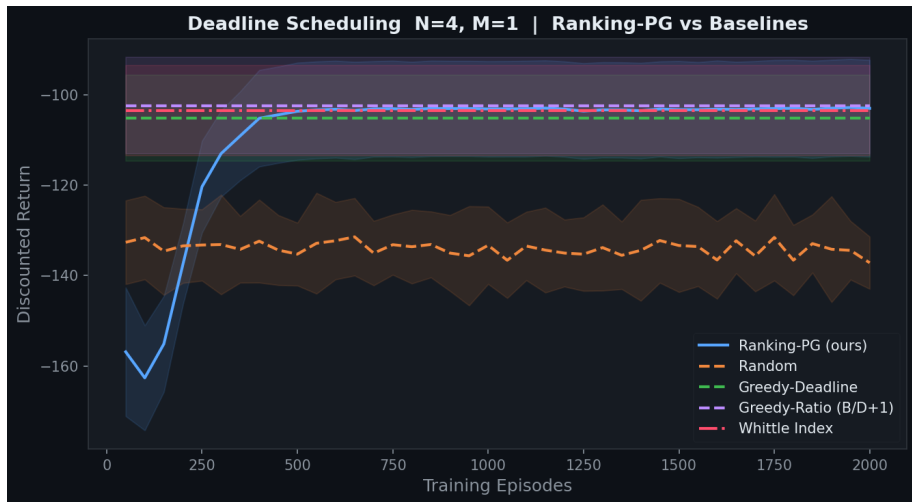


Figure: $N=4, M=1$

Results

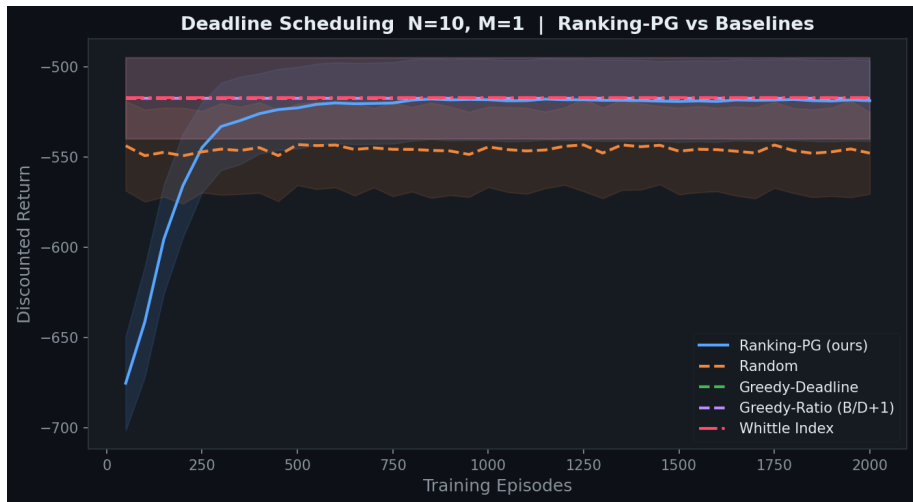


Figure: $N=10, M=1$

Results

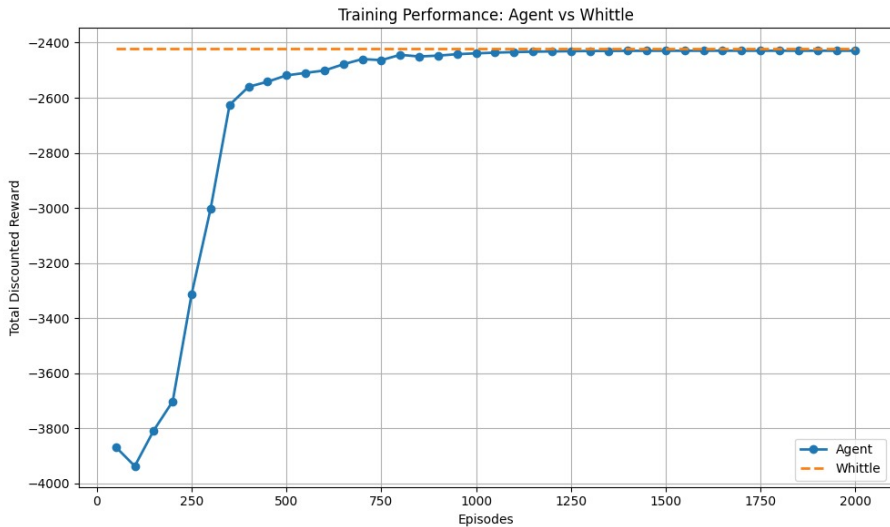


Figure: $N=100$, $M=25$

Do We Really Need PPO?

Experiment:

- Removed PPO clipping objective and trained using vanilla policy gradient.

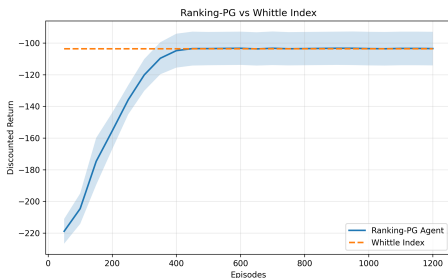


Figure: $N=4$, $M=1$

Observation: Convergence behavior was nearly unchanged. No major instability observed.

Machine Repair RMAB

Goal:

- We have N machines operating simultaneously
- Only M machines can be repaired at each timestep
- Need to decide *which machines to repair*

Machine state

Each machine has a discrete health state

$$s \in \{0, 1, \dots, S - 1\}$$

- $s = 0$: perfectly healthy
- $s = S - 1$: completely broken

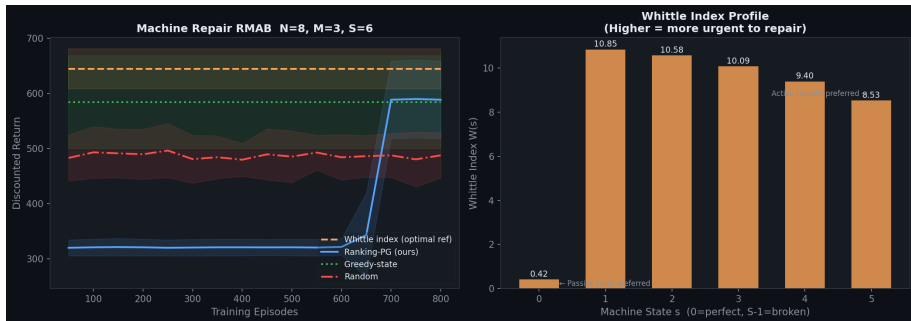
Reward function

Healthy machines produce higher reward:

$$r(s) = (S - 1) - s$$

Hence,

Results



- Ranking-PG substantially outperforms random scheduling
- Learned policy converges close to heuristic greedy scheduling
- Whittle policy remains best-performing reference
- Learned policy exhibits delayed performance jump, indicating late discovery of better ranking structure
- Whittle indices are not monotonic with degradation level:

$$W(1) > W(2) > W(3) > W(4) > W(5)$$

This suggests the optimal strategy prioritizes preventive maintenance rather than repairing already-failed machines.

Hence the RMAB policy is genuinely nontrivial.